SiteMind

A procurement agent that stays with a construction job for weeks.

It plans the job, calls specialist agents, survives `kill -9`, never sends the same RFQ or PO twice, and asks a human before anything that cannot be undone.

THE GOAL, AS TYPED

```
Get the 480 V switchgear package for
Project Alpha ordered by 2027-03-15
within budget ₹4.6 Cr.
```

30 simulated days, one run

39 checkpoints, one per task

1 SIGKILL mid-run, resumed

0 duplicate RFQs or POs

Track 01 · Amber General Intelligence

Challenge 1.3 · Work that takes days, not minutes

also shows 1.2 · the conductor

MIT licence

Demo vendors, prices and project data are synthetic

THE PROBLEM

On a data-centre job, one switchgear package sets the energisation date.

4–6
MONTHS LEAD TIME

A 480 V switchgear package is manufactured to order. The hall cannot be energised without it.

Weeks
OF EMAIL AND EXCEL

Procurement for that one package runs across mail threads and a tracker that someone has to keep current.

1
DROPPED STEP

A missed date slips energisation. A duplicated email or order costs money and credibility.

WHAT THOSE WEEKS CONTAIN, AND WHERE THEY BREAK

01	Takeoff from the spec	quantity disagrees with drawing
02	RFQs to several vendors	sent twice after a hand-over
03	Chase the ones that go quiet	follow-up nobody sends
04	Re-check prices before award	revision nobody re-budgets
05	Lead time vs required-on-site date	float found negative too late
06	Two sign-offs	approver on leave
07	Issue the purchase order	ordered twice, or not at all

Each step is easy. Staying with all of them for weeks, through restarts and hand-overs, is where the job fails.

WHAT SITEMIND DOES

One sentence in. A sent purchase order out, with a trail a project manager can follow.

> Get the 480 V switchgear package for Project Alpha ordered by 2027-03-15 within budget ₹4.6 Cr.

need-by 2027-03-15 | budget ₹4,60,00,000 ex-GST | required on site 2027-07-30

Project Alpha · 24 MW Tier-III data centre, Chakan MIDC, Pune · synthetic demo job

01
Plans
A 14-task dependency graph with deadlines, cost allowances and 5 risks.

02
Calls specialists
Takeoff, quotes, schedule (CPM), budget, approvals, PO. One per task.

03
Monitors
Checks the facts after every task and issues plan v2 when they break the plan.

04
Survives restarts
Resumes from the last checkpoint. A ledger stops repeated actions.

05
Asks a human
Ranked decision memo, then waits. Vendor choice and PO release are gated.

06
Issues the PO
Drafts the package, gets the second sign-off, sends it exactly once.

DEMO RUN OUTCOME
PO sent
sim 2027-02-10, to Anvil Electric India (synthetic vendor)

AGAINST THE ORDER-BY DATE
33 days early
need-by was 2027-03-15

PO VALUE
₹4,70,32,912
incl. 18 % GST · ₹3,98,58,400 ex-GST

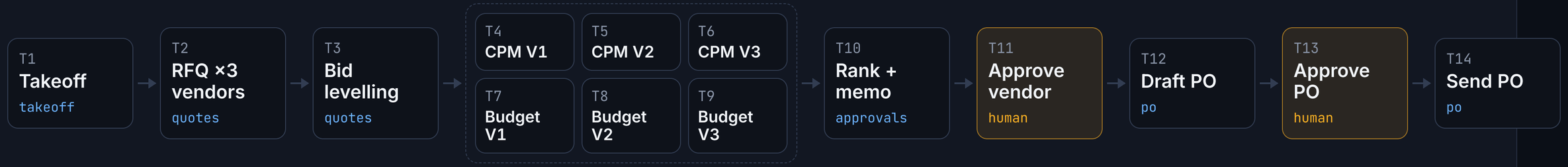
DELIVERY
2027-07-22
8 days ahead of the required-on-site date

HOW IT WORKS

A planner owns the task graph. Registered specialists do each step.

PLAN V1 · 14 TASKS · DEADLINES SCHEDULED BACKWARD FROM THE NEED-BY DATE

• specialist task • human approval



SPECIALIST REGISTRY · EACH IS ALSO AN MCP TOOL

SPECIALIST	TAKES	RETURNS	MCP TOOL
takeoff	spec PDF	lines with source, confidence, warnings	`takeoff_from_pdf`
quotes	lines, vendors, sim time	quotes and a per-attempt log	`request_vendor_quotes`
schedule	lead time, PO date, on-site date	critical path and float	`compute_schedule`
budget	quote, ceiling, contingency, fees	ok / warn / breach	`check_budget`
approvals	quotes, schedules, budget checks	ranked options, decision memo	`rank_vendor_options`
po	goal, chosen option, quote	purchase order package	`draft_purchase_order`

RUN LOOP · ONE TASK PER STEP

`plan` → `execute` → `monitor` → `replan` → `finish`

The monitor compares facts with the plan after every task. A trigger sends the run through `replan` and back to `execute`.

WHO DECIDES WHAT

- **Code** builds the graph and does every quantity, price, date and score.
- **The local model** words the rationale and the memo. It never produces a number.
- **Specialists are pure functions**, so a replay gives identical output.

DURABILITY

Kill the worker at any point. It resumes, and nothing is sent twice.

1 · CHECKPOINT PER TASK

The plan is saved after every task

LangGraph `StateGraph` with `SqliteSaver`. Each step runs exactly one plan task, then persists the plan, the facts and the simulated time.

The worker is its own OS process. The API and the worker share nothing in memory, so a dead worker loses nothing.

`thread_id = run_id` `SQLite WAL`

2 · IDEMPOTENCY LEDGER · ROWS FROM THE KILLED- REQUESTED AND-RESUMED DEMO RUN

<run>:T2-rfq:rfq:V1:a1	rfq.sent	×1
<run>:T2-rfq:rfq:V1:a2	rfq.sent	×1
<run>:T2-rfq:rfq:V1:a3	rfq.sent	×1
<run>:T3-levelling:refresh:V1:r1	quote.refresh	**×2**
<run>:T3-levelling:refresh:V2:r1	quote.refresh	**×2**
<run>:T3-levelling:refresh:V3:r1	quote.refresh	×1
<run>:T11-approve-vendor:approval:requested	approval	**×2**
<run>:T14-send-po:po:PO-ALPHA-20270207-V3-01	po.sent	×1

Every side effect goes through `perform_once(key, …)`. The key is the table's primary key. ×2 means requested by two workers and **executed once**: the second request got the stored result.

3 · KILL -9, THEN RESUME

`sim 01-17`	Worker A finishes the RFQ round. Checkpoint 7 written.
`SIGKILL`	Worker A dies inside `T3-levelling`, after two quote refreshes went out.
`resume`	Worker B starts from checkpoint 7 and replays T3 from its start.
`ledger`	The two refreshes return stored results. Only the third runs.

39
checkpoints in the run

17
ledger keys, each executed once

5
RFQs delivered: V1 ×3 attempts, V2, V3

1
PO delivered to the vendor outbox

STATED BOUNDARY

A worker that dies between an external effect and its ledger write retries that action; the outbox row carries the same key for the receiver to deduplicate.

FAILURE HANDLING

Three things go wrong in the demo. The agent recovers from each and re-plans once.

sim
01-11 → 01-17

V1 Voltaris

Vendor goes silent, twice

RFQ attempts 1 and 2 time out. Attempt 3 is answered: **₹3,54,75,400**, 140 days lead.

`retry ×3` Each attempt has its own ledger key and audit entry. No re-plan needed.

sim
01-18 → 01-22

V2 Kestrel

Price moves after the quote

Revision 2 adds an 11 % raw-copper surcharge and withdraws the 3 % early-order discount: **₹3,86,51,400 → ₹4,41,66,800** (+14.3 %). With 5 % contingency that is ₹4,63,75,140, **100.8 % of budget**, ₹3,75,140 over.

`budget_breach:V2` Negotiate. V2 holds its price, so the option is ruled out.

sim
01-21 → 01-28

V3 Anvil

Lead time breaks the on-site date

168 days lead puts standard delivery on 2027-08-29, **30 days late**. No in-budget option keeps 14 days of float.

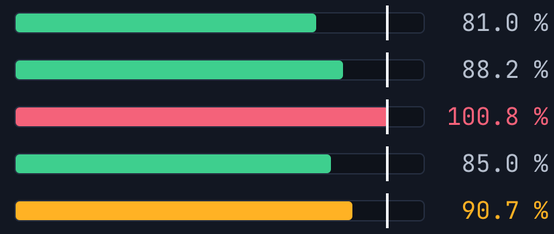
`deadline_break:V3` Evaluate Anvil's expedite offer: 42 days saved for ₹26,00,000. New option **O-V3X** lands inside the date at 90.7 % of budget.

PLAN V1 → V2 · SIM 2027-01-22

```
+ T15-negotiate-V2
+ T16-expedite-V3
+ T17-rank (new decision memo)
+ T18-escalate (notify the PM)
~~T10-rank~~
~ T11-approve-vendor now waits on T18
```

The monitor waits until every schedule and budget check is in, so both triggers produce one new plan. Final plan: 18 tasks, 17 done, 1 superseded and kept as `skipped`.

COMMITTED COST AS % OF THE ₹4.6 CR CEILING



White line = 100 % of budget. Quote + 5 % contingency + fees, ex-GST. Warn from 90 %, breach above 100 %.

HUMAN IN THE LOOP · AUDIT TRAIL

Before anything irreversible, the agent writes a memo and stops.

DECISION MEMO · SIM 2027-01-29 · PLAN V2 — waiting for approval

OPTION	BUDGET USE	DELIVERY	FLOAT	ON-TIME	RESULT
O-V3X Anvil, expedited	90.7 %	2027-07-24	+6 d	90 %	**recommended**
O-V2 Kestrel, rev 2	100.8 %	2027-07-10	+20 d	93 %	over budget
O-V1 Voltaris	81.0 %	2027-08-07	−8 d	82 %	late
O-V3 Anvil, standard	85.0 %	2027-09-04	−36 d	90 %	late

RISKS THE MEMO LISTS AGAINST ITS OWN RECOMMENDATION

- 6 days of float is thin.
- The option sits above the 90 % budget warning line.
- The ₹26 L expedite fee should be payable only against a delivery milestone with LDs.

TWO APPROVAL GATES · NO PROCESS RUNS WHILE THE AGENT WAITS

memo → approve vendor → draft PO → approve PO release → send once

The person can approve, pick another option, or reject. A rejection is a trigger: the planner re-ranks without that option and asks again.

AUDIT TRAIL · 36 ENTRIES IN THE DEMO RUN

01-11 17:00	takeoff	8 line items from the spec PDF, confidence 0.92. RFI raised: schedule lists 4 SPDs, drawing p3 shows 3.
01-15 23:00	quotes	V1 silent after 2 days (attempt 2/3), retrying.
01-17 23:00	system	Worker killed (SIGKILL) at T3-levelling. Resumed from checkpoint 7.
01-22 11:00	planner	Re-plan v2: +4 tasks, 1 superseded.
01-29 14:00	approvals	Recommend O-V3X. 1 of 4 options feasible. Escalated to PM.
02-04 02:00	human	Approved O-V3X Anvil Electric India.
02-10 02:00	po	PO-ALPHA-20270207-V3-01 release approved and sent.

EVERY ENTRY RECORDS

`actor` `action` `reasoning summary` `inputs` `outputs` `cost` `confidence` `plan version`

Each takeoff line cites its source, for example `…switchgear_spec.pdf p4 table 1 row 1`.

ARCHITECTURE · STACK

Two SQLite files hold the whole run. Everything else can be restarted.

React dashboard
Vite, TypeScript, Tailwind. Polls every second.

→

FastAPI `:8000`
Creates runs, records approvals, spawns and kills workers.

→

Worker process `python -m sitemind.runner <run_id>`
One OS process per active run, with a per-run file lock.

`plan` → `execute` → `monitor` → `replan` → `finish`

LangGraph `StateGraph`. `interrupt()` parks the run for a human; `Command(resume=…)` continues it.

Amber, or any MCP client
Calls the specialists directly.

→

MCP server
Official Python SDK. stdio or streamable HTTP. 7 tools.

Specialist registry

`takeoff` `quotes` `schedule` `budget` `approvals` `po`

Pure Python. No clock, network or model inside. Quotes come from a mock vendor network (synthetic scenario file).

SQLite, WAL mode

`checkpoints.sqlite`
LangGraph SqliteSaver, one checkpoint per task

`sitemind.sqlite`
runs, audit, ledger, approvals, timeline, outbox

Ollama · `qwen2.5:3b` Local, temperature 0, wording only. Each call is cached in the ledger. If the model is unreachable a template is used and the audit entry says so.

STACK

agent	LangGraph + langgraph-checkpoint-sqlite
contract	Pydantic v2 models, JSON in and out
api	FastAPI, Uvicorn
ui	React 18, Vite, Tailwind
agent-to-agent	MCP Python SDK
model	Qwen2.5-3B-Instruct on Ollama, via pydantic-ai
documents	pdfplumber, ReportLab

₹0
API fees per run

3
model calls per run

5.4 s
full run, model off, no pacing

1
command: `docker compose up --build`

IMPACT · JUDGING CRITERIA

Built against what challenge 1.3 asks for, and what the judges score.

40 %

Accuracy

Does it get the job right on realistic examples?

- Every quantity, price, date and score comes from deterministic code and is reproduced by the unit tests.
- A killed and resumed run ends with the same PO as an uninterrupted one.

30 %

Customer value

Would a procurement manager use it tomorrow?

- It does the chasing, re-checking and escalating a coordinator does by hand.
- The demo PO goes out 33 days ahead of the order-by date, with a memo a PM can sign.

20 %

Trustworthiness

Sources cited, sensible confidence, nothing made up.

- Takeoff lines cite page, table and row. Confidence 0.92, with an RFI where the drawing disagrees.
- The memo lists the risks of its own recommendation. Synthetic data is labelled everywhere.

10 %

Build quality

Easy to run, cost and speed per run, code quality.

- One command to start. ₹0 per run on a laptop CPU. 5.4 s with the model off.
- Typed contract, deterministic test suite with kill-and-resume tests, MIT licence.

CHALLENGE 1.3 · “GOOD LOOKS LIKE”

“recovers from setbacks on its own where it can”
`3 RFQ attempts, negotiation, expedite, plan v2`

“never does the same thing twice”
`17 ledger keys, 5 RFQs and 1 PO, each once`

“picks up correctly after a restart”
`SIGKILL, resume from checkpoint 7`

“leaves a trail a project manager can follow”
`36 audit entries, versioned plan, timeline`

REQUIRED OF EVERY ENTRY

- **Structured output.** Pydantic models, JSON throughout.
- **Callable by another agent.** MCP server, 7 tools.
- **Sources and confidence.** On every takeoff line and audit entry.
- **Hand-off to a human.** Two approval gates.

Sterling · Criteria and weights from the challenge Resources page, Round 1

WHAT'S NEXT · CREDITS

The harness is the product. Procurement is the first job on it.

NEXT

01 **Email and ERP adapters** behind the same ledger, passing the idempotency key to the receiving system.

02 **More job types** on the same harness: submittal chasing, RFI follow-up, delivery tracking.

03 **Lead-time ranges** from vendor history in place of single values.

04 **Takeoff from real drawing sets**, with a review screen where a person accepts each line.

05 **Postgres, authentication, approver roles** and delegation.

06 **A kill-at-every-step benchmark** that asserts zero duplicate actions across the run.

LIMITS TODAY

Synthetic vendors and prices. Simulated clock. RFQs and POs go to a mock outbox, with no real email or ERP. One package type and one spec format. Single tenant.

BUILT ON OPEN SOURCE

LangGraph + checkpoint-sqlite	MIT	durable graph, checkpoints, interrupt()
MCP Python SDK	MIT	specialists as MCP tools
Pydantic v2 · Pydantic AI	MIT	data contract, structured model output
FastAPI · Uvicorn · HTTPX	MIT / BSD-3	REST API
Ollama	MIT	local model serving
Qwen2.5-3B-Instruct	Qwen Research	wording only; non-commercial licence
React · Vite · Tailwind	MIT	dashboard
pdfplumber · ReportLab	MIT / BSD-3	read and generate the sample spec PDF

IDEAS BORROWED · NO CODE COPIED

bytedance/deer-flow	MIT	lead agent plus specialist sub-agents
Kentucky-ai/opentakeoff	AGPL-3.0	takeoff as an agent tool, source on every line
DDC Skills for AI Agents	MIT	construction workflow vocabulary

SiteMind is original work under the MIT licence. Full list with links in `CREDITS.md`.